

Toegepaste Informatica

3TX

2025-2026



UCLL
HOGESCHOOL



Full-stack Software Development

Back-end: Recap

R. Naudts, J. Pieck, J. Rombouts, B. Van Impe

Overview

- We're going to review
 - Spring boot
 - HTTP
 - Layered architecture: each layer and the responsibility of each layer
 - JPA relationships
 - Testing

Part 1: Spring boot

Spring Boot

- Spring Boot is
 - A backend framework that helps with typical backend functionality
 - Starting and stopping a server that can respond to HTTP requests
 - Retrieving and storing data from a database
 - Securing your backend
 - ...
 - A modular framework
 - Every module can be loaded separately by specifying different dependencies
 - A high-level framework
 - It loads in many more low-level dependencies to provide its functionality



Spring Initializr: get started quickly

The screenshot shows the Spring Initializr web application interface. The browser address bar displays 'start.spring.io'. The page features a sidebar with a hamburger menu and a clock icon. The main content area is divided into several sections:

- Project:** Includes radio buttons for 'Gradle - Groovy', 'Gradle - Kotlin', 'Java' (selected), and 'Groovy'. Below this is a 'Maven' link.
- Spring Boot:** Includes radio buttons for '4.0.0 (SNAPSHOT)', '4.0.0 (M3)', '3.5.7 (SNAPSHOT)', '3.5.6' (selected), and '3.4.11 (SNAPSHOT)'. Below this is a '3.4.10' link.
- Project Metadata:** Includes input fields for 'Group' (be.ucll.fullstack), 'Artifact' (demo), 'Name' (demo), 'Description' (Demo project for Spring Boot), and 'Package name' (be.ucll.fullstack.demo).
- Packaging:** Includes radio buttons for 'Jar' (selected) and 'War'.
- Language:** Includes radio buttons for 'Java' (selected), 'Kotlin', and 'Groovy'.
- Dependencies:** Includes a button 'ADD DEPENDENCIES... CTRL + B'. Below this are three sections:
 - Spring Web:** Labeled 'WEB', description: 'Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.'
 - Spring Data JPA:** Labeled 'SQL', description: 'Persist data in SQL stores with Java Persistence API using Spring Data and Hibernate.'
 - Spring Boot DevTools:** Labeled 'DEVELOPER TOOLS', description: 'Provides fast application restarts, LiveReload, and configurations for enhanced development experience.'

At the bottom of the page, there are three buttons: 'GENERATE CTRL + G', 'EXPLORE CTRL + SPACE', and a button with three dots.

Part 2: HTTP

HTTP Methods

- **GET:** Retrieves data from a specified resource
- **POST:** Submits data to a specified resource
- **PUT:** Stores or updates a resource at a specified location
- **DELETE:** Requests the removal of a specified resource
- **PATCH:** Applies partial modifications to a resource

Idempotent?

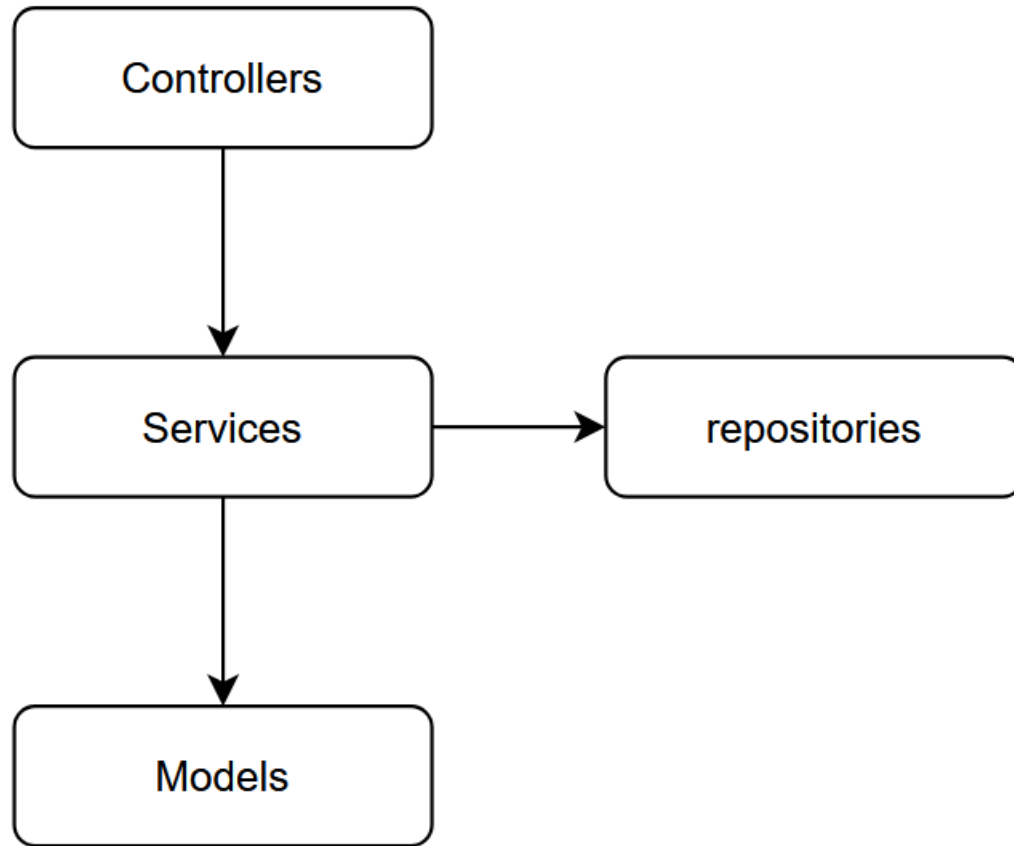
- making the same request multiple times has the same effect as making it once.
- Idempotent: GET, DELETE and PUT
- Not idempotent: POST, PATCH
 - It is allowed for a post or a patch request to have side effects

HTTP Status Codes

- The status code of the response indicates whether the backend could successfully process the request
 - 2xx – Success
 - 3xx – Redirect
 - 4xx – Client Error
 - 5xx – Server Error
- Other, more specialised status codes exist
 - https://en.wikipedia.org/wiki/List_of_HTTP_status_codes

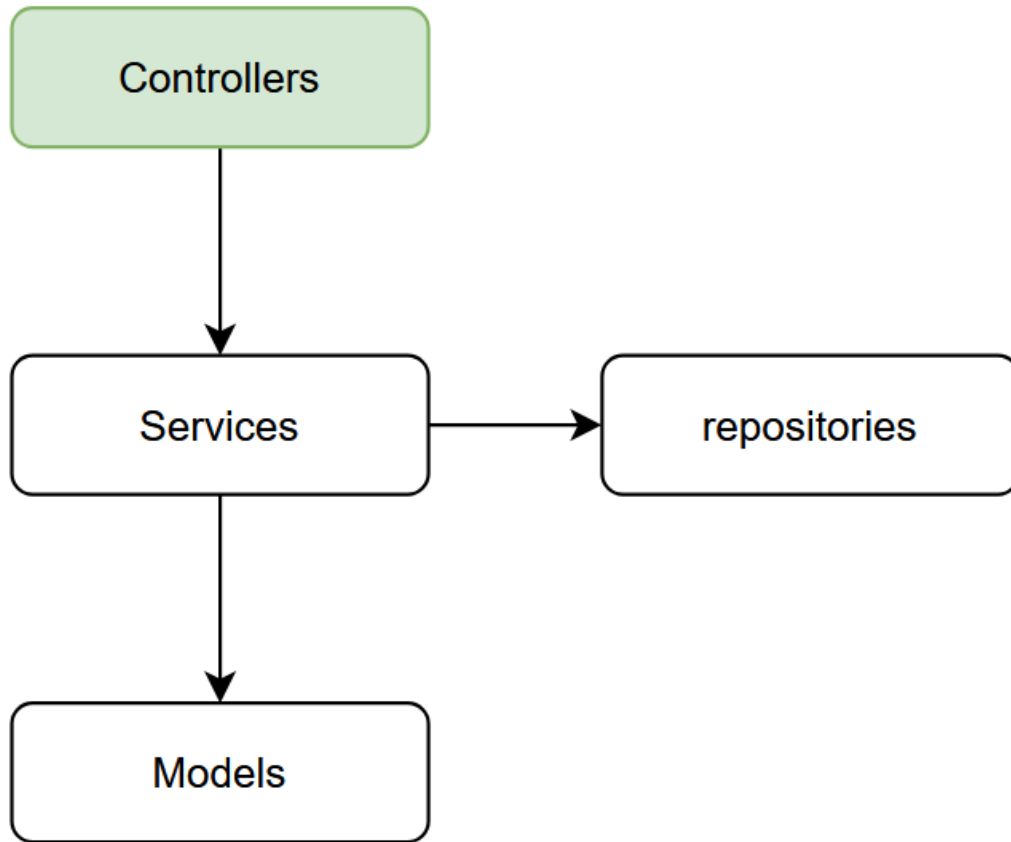
Part 3: layered architecture

Layered Architecture



- Our backend is divided in different layers, all with their own responsibilities
- As a general rule: layers have no knowledge about the layers above them

Layered Architecture: Controllers



- Responsibilities:
 - Listening to http requests
 - Deserializing input from that request
 - Calling the service layer to handle the call
 - Serializing the output (result or error messages) to send back to the caller

Controller method example

```
@RestController  Jeroen Rombouts *
@RequestMapping("/pony")
public class PonyController {
    |
    @PostMapping  Jeroen Rombouts
    public Pony addPony(@Valid @RequestBody Pony pony) {
        return ponyService.addPony(pony);
    }
}
```

This method handles post requests done to the /pony url

This controller deserializes the json in the request body to make a pony object. The validation is done now.

The method seems to return a pony, but the response gets serialized to json before it is returned to the caller.

The controller method calls the service to handle the business logic.

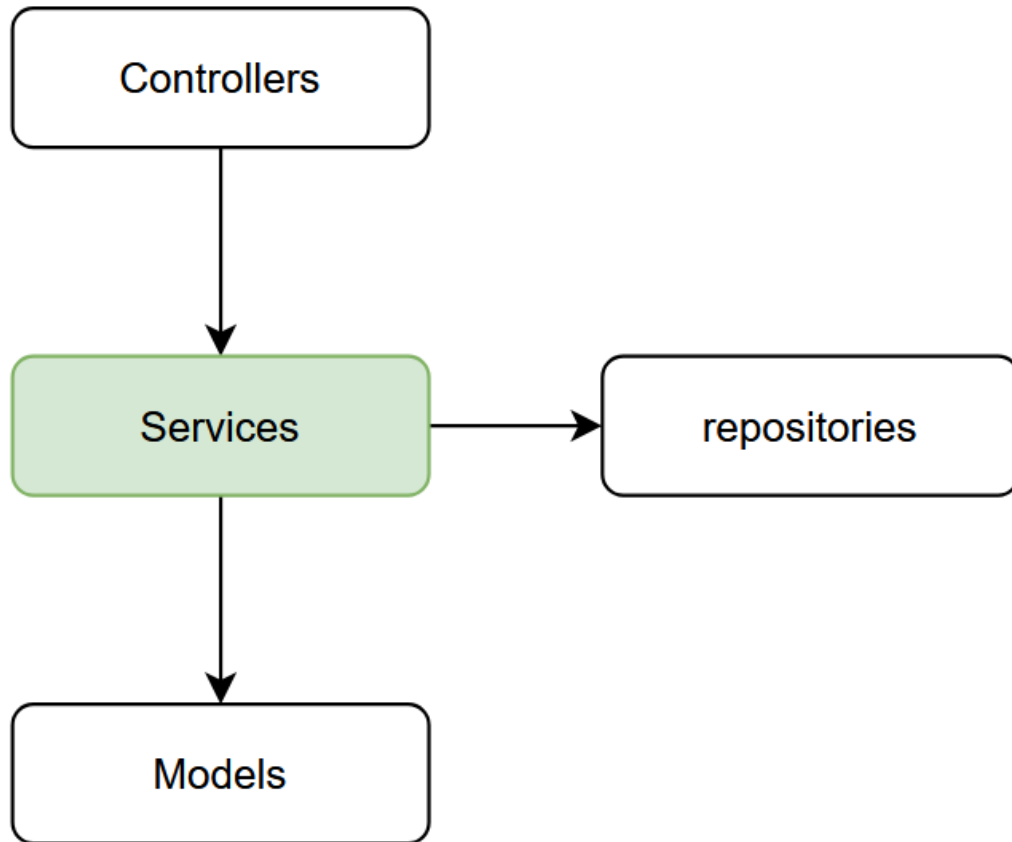
Controller method example

```
@RestController  Jeroen Rombouts *  
@RequestMapping("/pony")  
public class PonyController {  
    |  
    @PostMapping  Jeroen Rombouts  
    public Pony addPony(@Valid @RequestBody Pony pony) {  
        return ponyService.addPony(pony);  
    }  
}
```

The @Valid annotation indicates that the type has validations in its class definition.

```
public class Pony extends Animal {  
  
    @NotBlank(message = "name may not be empty") 3 usages  
    private String name;  
  
    @Positive(message = "age must be positive") 3 usages  
    private int age;  
}
```

Layered Architecture: Services

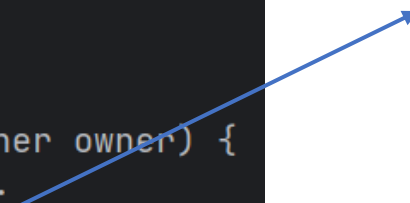


- Responsibilities:
 - Centralizing business logic
 - Optionally: Handle transactions
 - Handle integration with other parts of the system, eg:
 - Database via repositories
 - Sending messages on a message queue
 - Calling another api

Service method example

```
@Service 6 usages Jeroen Rombouts *  
public class PonyService {  
    // parts snipped for brevity  
    public Pony addOwner(String name, Owner owner) {  
        Pony pony = findPonyByName(name);  
        pony.setOwner(owner);  
  
        ownerRepository.save(owner);  
        return ponyRepository.save(pony);  
    }  
}
```

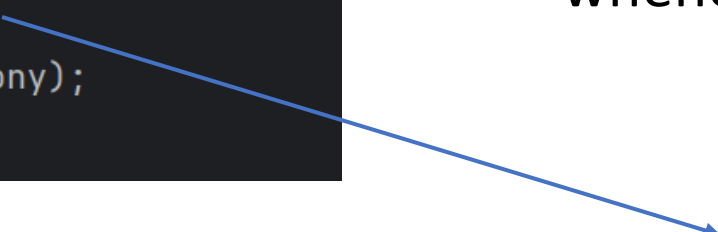
Call repository to get data



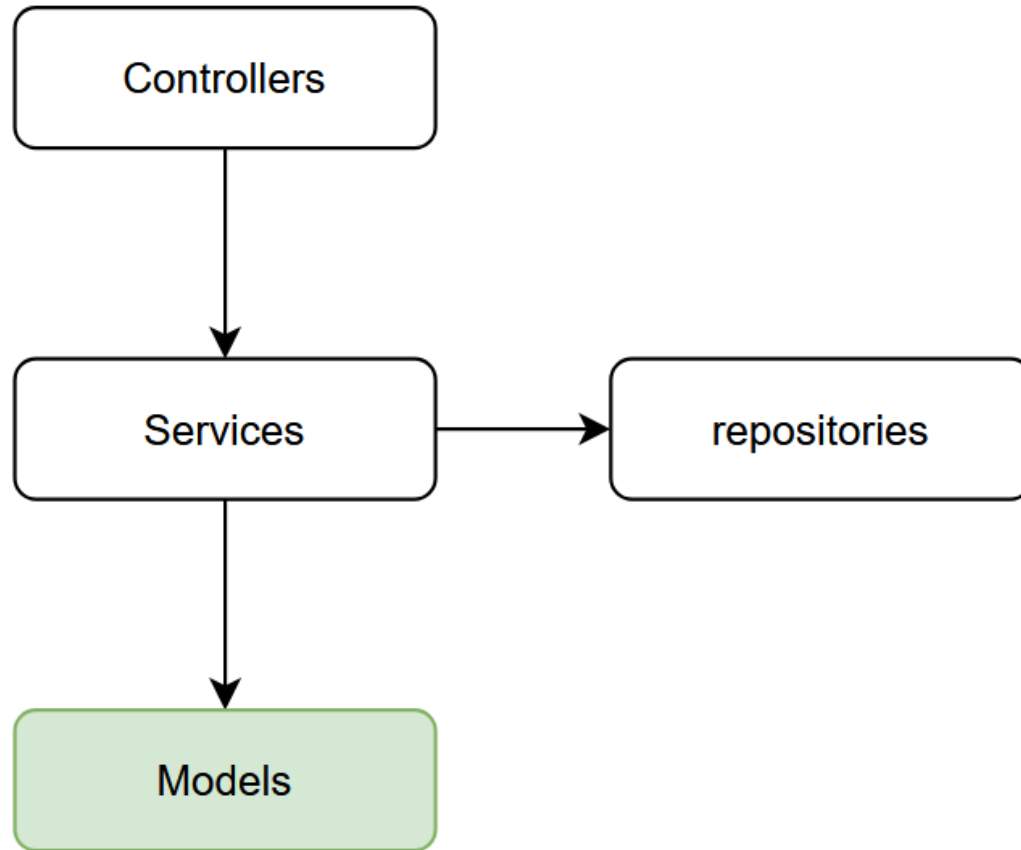
Delegates to the models
whenever possible



calls repositories in the right
order and creates a return value



Layered Architecture: Models



- Responsibilities:
 - Implements the core business logic and business rules for the application

Model example

```
@Entity 60 usages Jeroen Rombouts
@DiscriminatorValue("pony")
public class Pony extends Animal {

    @NotBlank(message = "name may not be empty") 3 usages
    private String name;

    @Positive(message = "age must be positive") 3 usages
    private int age;

    @OneToOne 2 usages
    @JoinColumn(name = "owner_id")
    private Owner owner;

    // For JPA, not public on purpose
    protected Pony() {} Jeroen Rombouts

    public Pony(String name, int age) { 5 usages Jeroen Rombouts
        this.name = name;
        this.age = age;
    }

    public String getName() { return name; }

    public int getAge() { return age; }

    public void updateNameAndAge(String name, int age) { 1 usage
        this.name = name;
        this.age = age;
    }

    public void setOwner(Owner owner) { this.owner = owner; }

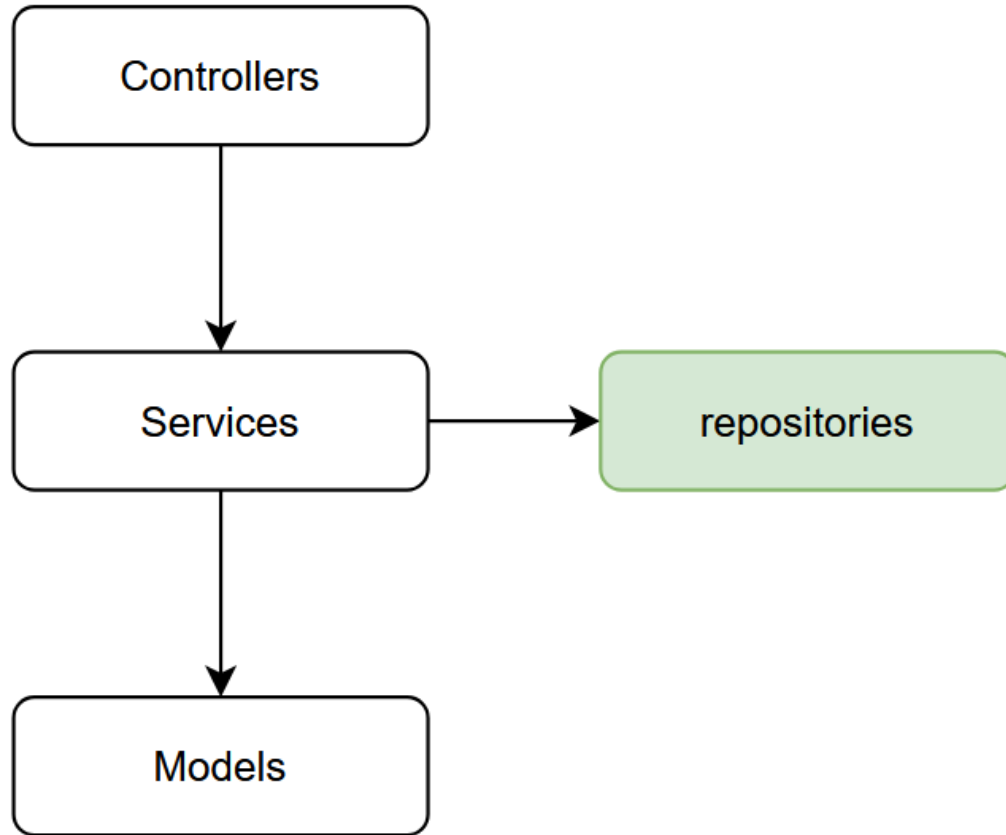
    public Owner getOwner() { return owner; }
}
```

Validation Rules: when is an object valid

Describes relationships between 2 classes

Describes business rules, like how to add an owner to a Pony.

Layered Architecture: Repositories



- Responsibilities:
 - To abstract and encapsulate data access logic

Layered Architecture: Repositories

```
public interface OwnerRepository extends JpaRepository<Owner, Long> {  
    Optional<Owner> findByNameContainingOrderByname(String name);  
}
```

Determines type and type of the key

Already implements several repository methods, like `findAll()`, `save(owner)`, `exists(owner)`, etc

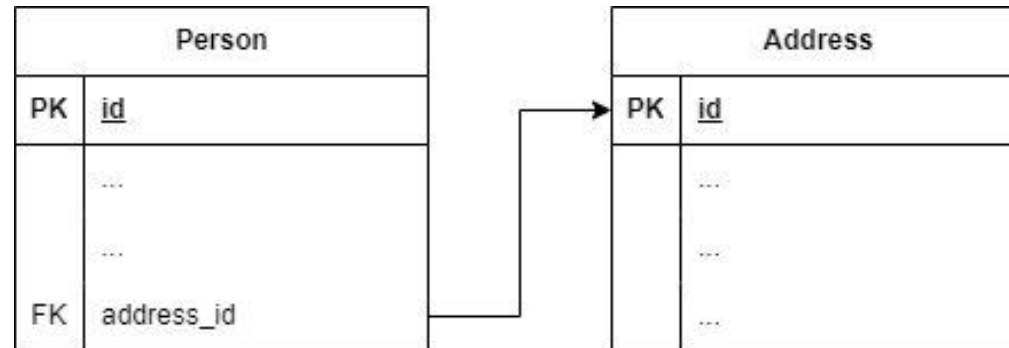
And we can use keywords to build our own query method

For more: <https://docs.spring.io/spring-data/jpa/reference/jpa/query-methods.html>

Part 4: JPA relationships

JPA review: ownership

- Describing relationships between @Entities
- Who owns the relationship:



The table containing the Foreign key is the owner

JPA review: direction

- Describing relationships between @Entities
 - Unidirectional: can only be navigated in one direction, there is only one place to add mapping

```
@Entity
@Table(name = "person")
public class Person {

    @OneToOne
    @JoinColumn(name = "address_id")
    private Address address;
}
```

```
@Entity
@Table(name = "address")
public class Address {

    // ...
}
```

- Bidirectional: can be navigated from both sides, requires mapping on both sides

```
@Entity
@Table(name = "cart")
public class Cart {

    @OneToMany(mappedBy = "cart")
    private List<CartItem> items;
}
```

```
@Entity
@Table(name = "cart_item")
public class CartItem {

    @ManyToOne
    @JoinColumn(name = "cart_id")
    private Cart cart;
}
```

JPA review: Unidirectional mapping

- Unidirectional one-to-many:

```
@Entity no usages
public class Department {

    @Id
    private Long id;

    @OneToMany no usages
    @JoinColumn(name = "department_id")
    private List<Employee> employees;
}
```

JPA review: Unidirectional mapping

- Unidirectional many-to-one:

```
@Entity no usages
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name; no usages

    @ManyToOne no usages
    @JoinColumn(name = "school_id")
    private School school;
}
```

JPA review: Unidirectional mapping

- Unidirectional one-to-one:

```
@Entity 1 usage
public class Employee {

    @Id
    private Long id;

    @OneToOne no usages
    @JoinColumn(name = "parking_spot_id")
    private ParkingSpot parkingSpot;
}
```

JPA review: Unidirectional mapping

- Unidirectional many-to-many:

```
@Entity no usages
public class Book {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String title; no usages

    @ManyToMany no usages
    @JoinTable(name = "book_author",
        joinColumns = @JoinColumn(name = "book_id"),
        inverseJoinColumns = @JoinColumn(name = "author_id"))
    private Set<Author> authors;

}
```

JPA review: Bidirectional mapping

- Bidirectional one-to-many / many-to-one

```
@Entity 1 usage
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne no usages
    @JoinColumn(name = "department_id")
    private Department department;
}
```

owner-side: @JoinColumn: field that maps to the column with the foreign key

```
@Entity 1 usage
public class Department {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @OneToMany(mappedBy = "department") no usages
    private List<Employee> employees;
}
```

Non-owner-side: mappedBy: the field that has the relationship on the owner side

JPA review: Bidirectional mapping

- Bidirectional many-to-many :

```
@Entity 1 usage
public class Course {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToMany no usages
    @JoinTable(name = "course_student",
        joinColumns = @JoinColumn(name = "course_id"),
        inverseJoinColumns = @JoinColumn(name = "student_id"))
    private List<Student> students;
}
```

owner-side: @jointable: define the name of the jointable
@JoinColumns and @InverseJoinColumns define the names of the columns

```
@Entity 1 usage
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToMany(mappedBy = "students") no usages
    private List<Course> courses;
}
```

Non-owner-side: mappedby: the field that has the relationship on the owner side

JPA review: Bidirectional mapping

- Bidirectional many-to-many :
- Who is owner of a bidirectional many-to-many? Neither side has the PK of the other.

=> you can choose.

JPA review: Bidirectional mapping

- Bidirectional mapping: avoid infinite Json recursion:
- **@JsonManagedReference**: Represents the *forward reference* in a relationship. Fields annotated with this are serialized normally.
- **@JsonBackReference**: Represents the *backward reference* in a relationship. Fields annotated with this are omitted from serialization.

Part 4: Testing

Testing: unit testing domain models

- Writing a Junit test for this simple class:

```
package be.ucll.fullstack.relationships.model;

public class Calculator { no usages

    private int currentValue = 0; 2 usages

    public int AddToCurrentValue(int addValue) { no usages
        if(addValue < 0) {
            throw new IllegalArgumentException("addValue must be positive");
        }
        currentValue += addValue;
        return currentValue;
    }
}
```

Testing: unit testing domain models

```
public class CalculatorTest {  
    private Calculator calculator; 3 usages  
  
    @BeforeEach  
    void initTest(){  
        calculator = new Calculator();  
    }  
  
    // happy path  
    @Test  
    void givenNewCalculator_whenAdding2_thenResultIs2(){  
        var result = calculator.AddToCurrentValue( addValue: 2);  
        Assertions.assertEquals( expected: 2, result );  
    }  
  
    // testing exception  
    @Test  
    void givenNewCalculator_whenAddingNegativeNumber_thenThrowIllegalArgumentException(){  
        var exception = Assertions.assertThrows(IllegalArgumentException.class, () -> {  
            calculator.AddToCurrentValue( addValue: -2);  
        });  
        Assertions.assertEquals( expected: "addValue must be positive",  
            exception.getMessage());  
    }  
}
```

→ @BeforeEach: this method runs before each test method

→ @Test: indicates a test method

→ Use Assertions methods to check the results of method calls

→ Use this syntax to test that the system under test (sut) throws the correct Exception with the correct Exception message

Testing: unit testing Services

- Writing a Junit test for this service:

```
@Service 6 usages  Jeroen Rombouts *
public class PonyService {

    private PonyRepository ponyRepository; 8 usages
    private OwnerRepository ownerRepository; 2 usages

    @Autowired  Jeroen Rombouts
    public PonyService(PonyRepository ponyRepository, OwnerRepository ownerRepository) {
        this.ponyRepository = ponyRepository;
        this.ownerRepository = ownerRepository;
    }

    public Pony addPony(Pony pony) { 1 usage  Jeroen Rombouts
        return ponyRepository.save(pony);
    }
}
```

Testing: unit testing Services

- We need to mock the 2 repositories (Mockito) and inject them in the service:

```
@ExtendWith(MockitoExtension.class)
public class PonyServiceTest {

    @Mock 3 usages
    private PonyRepository ponyRepository;

    @Mock no usages
    private OwnerRepository ownerRepository;

    @InjectMocks 2 usages
    private PonyService ponyService;
```

Testing: unit testing Services

```
@Test
public void givenPony_whenSaving_thenCallsPonyRepoSave() {
    var bella = new Pony( name: "Bella", age: 5);
    Mockito.when(ponyRepository.save(bella)).thenReturn(bella);

    var resultPony = ponyService.addPony(bella);
    Mockito.verify(ponyRepository).save(bella);
    Assertions.assertEquals(bella, resultPony);
}
```

Tell Mockito what the mocked repo should do when the save method is called

Call the actual method we want to test

Verify that the repository's save method has been called by the service

Testing: unit testing Services

```
@Test
public void givenPony_whenSearchingByExistingName_thenCallsPonyRepoSave() {
    Mockito.when(ponyRepository.findPonyByName(Mockito.anyString()))
        .thenReturn(new Pony( name: "Bella", age: 5));

    var resultPony = ponyService.findPonyByName("Bella");
    Mockito.verify(ponyRepository).findPonyByName("Bella");
    Assertions.assertEquals( expected: "bella", resultPony.getName().toLowerCase());
}
```

Return the same value no matter
the search string



Testing: end-to-end testing

```
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
@AutoConfigureWebTestClient
public class PonyEndToEndTestClass {
    @Autowired
    private WebTestClient client;

    @Autowired
    private PonyRepository repository;

    @Test
    public void givenNoPonyTornado_whenInvokingPostTornado_thenPonyTornadoIsSaved() {
        client.post().uri("/pony")
            .header("Content-Type", "application/json")
            .bodyValue("{\"name\":\"Tornado\",\"age\":17}")
            .exchange()
            .expectStatus().is2xxSuccessful()
            .expectBody().json("{\"name\":\"Tornado\",\"age\":17}");

        Assertions.assertEquals("Tornado", repository.findPonyByName("Tornado").getName());
    }
}
```

The WebTestClient to call the api

Inject the repo to check the final result

Exchange: actually do the call

Don't forget to reset your data with @BeforeEach or @AfterEach